

Container Runtimes: Docker, CRI-O, containerd

Container sind neu, Container ersetzen Virtualisierung, Container sind prinzipiell sicher,
Container heissen Docker

(Einige der grössten Fehlannahmen im Hinblick auf das Thema Container und Docker)



Lernziele

- Sie haben einen Überblick über Container Runtimes wie Docker, CRI-O, containerd.

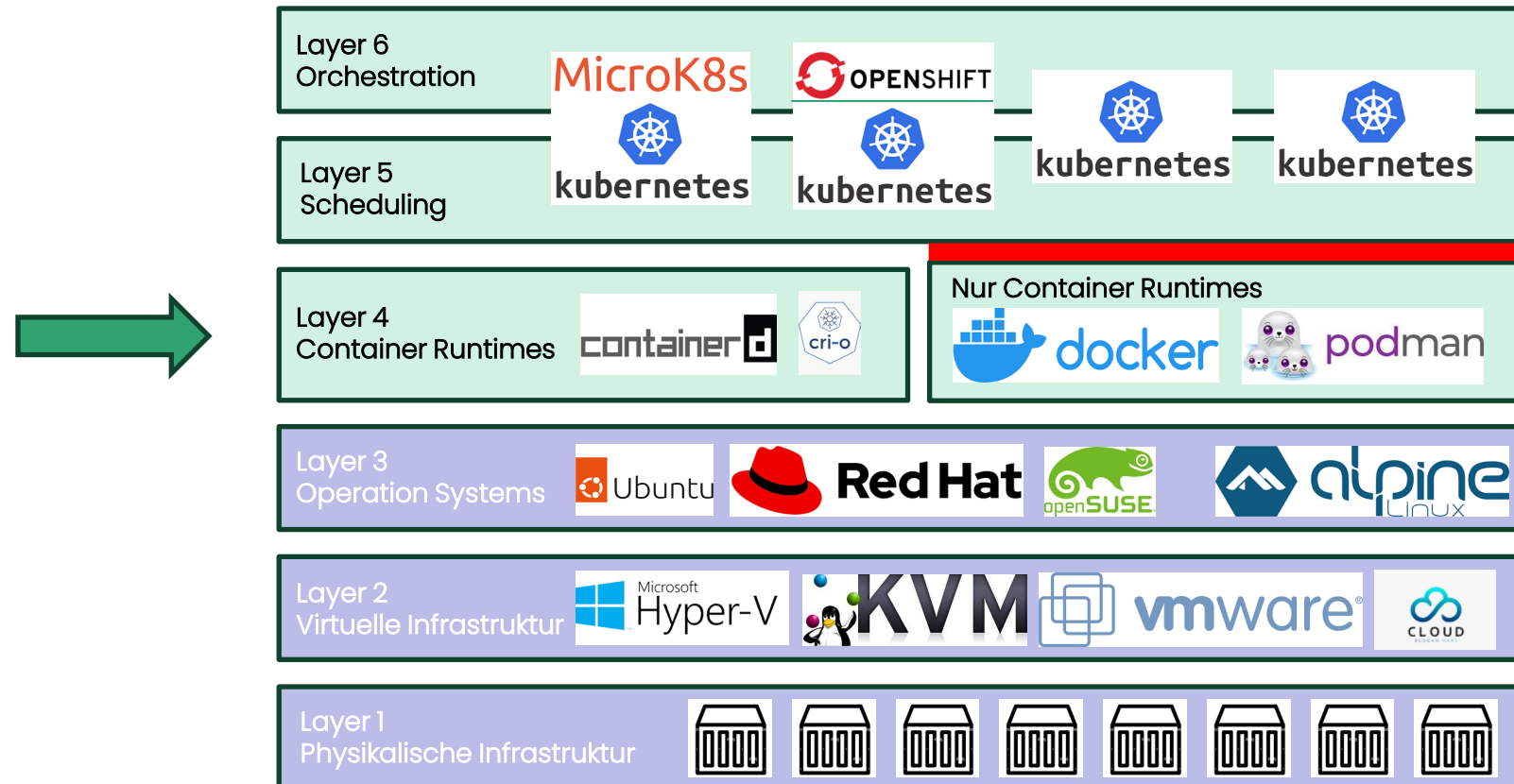
Zeitlicher Ablauf

- Container Runtimes
- Docker Versionen, wichtige Meilensteine und Inkompatibilitäten
- Funktionaler Überblick
- Wie kommen wir zu Container Images?
- Container Images bauen (u.a. docker build) und verwalten
- Docker Networking
- Docker Compose
- Docker Volumes
- Reflexion
- Lernzielkontrolle

Container – Geschichte

- Container-Technologien haben ihre Wurzeln in den 1970er-Jahren. 1979 führte UNIX mit *chroot* erstmals Dateisystem-Isolation ein.
- 1998 folgten *FreeBSD Jails*, die auch Prozesse isolierten.
- 2001 kamen *Solaris Zones* sowie *Virtuozzo* (später als *OpenVZ*) hinzu, die Container auf Linux brachten.
- Google entwickelte anschliessend *cgroups* zur Ressourcenbegrenzung und legte damit die Basis für moderne Container.
- Das LXC-Projekt (2008) vereinte *cgroups*, *Namespaces* und *chroot* zu einer vollständigen Containerlösung
- Mit **Docker** (2013) wurde Containerisierung schliesslich massentauglich: einfach, portabel und effizient.
- Heute steht **Kubernetes** für die Orchestrierung grosser Containerlandschaften, während reine Docker-Umgebungen zunehmend an Bedeutung verlieren.

Container Runtimes und Kubernetes



Die (vereinfachten) »Layer« der Container-Welt

Container Runtimes

- Open Container Initiative (und damit Kubernetes) compatible Container Runtimes sind, u.a.:
 - ◇ **Docker (2013)**, im 2017 Marktanteil von 90 %.
-> Kubernetes is **deprecating Docker** as a container runtime after v1.20 (1.1.2021)!
 - ◇ **containerd** ist als Daemon für Linux und Windows verfügbar. Es verwaltet den gesamten Container-Lebenszyklus seines Host-Systems, von der Image-Übertragung und -Speicherung über die Container-Ausführung und -Überwachung bis hin zum Low-Level-Storage und Netzwerk.
-> **Es ist aus Docker entstanden, ist vereinfacht dessen Container Runtime.**
-> **Adaptors: IBM Cloud, Google, CloudFoundry, Docker, etc. -> CNCF graduated project**
 - ◇ **CRI-O**: das 2016 gestartete und 2017 vorgestellte CRI-O baut auf dem Kubernetes Container Runtime Interface (CRI) auf. CRI-O erlaubt das direkte Verwenden von OCI-kompatiblen (Open Container Initiative) Containern in Kubernetes, ohne zusätzlichen Code oder weitere Tools.
-> **Contributors sind RedHat, Intel, SUSE, IBM. -> CRI-O is a CNCF incubating project.**
-> **1.19 K8s Version: Empfehlung CRI-O Container statt Docker EE auf Windows verwenden!**
-> siehe auch: <https://www.linux-magazin.de/ausgaben/2020/09/podman/>

Docker: Versionen, wichtige Meilensteine und Inkompatibilitäten

- 1.9: mit komplett überarbeitetem Netzwerk-Modul
- 1.10: neues DB-Format
- 1.11: mit Support für runC und **containerD**
- 1.12: mit integriertem Swarm Mode
- 1.13: unter anderem mit neuem Plugin-Format (z.B. für Erweiterungen wie Volume Manager)



✓ For full Kubernetes Integration

- [Kubernetes on Docker for Mac](#) is available in 17.12 Edge (mac45) or 17.12 Stable (mac46) and higher.
- [Kubernetes on Docker for Windows](#) is available in 18.02 Edge (win50) and higher edge channels only.

- **Docker CE (Community Edition)**: Support für 4 Monate
- **Docker EE (Enterprise Edition)**: Support für ein Jahr ([Mirantis](#))

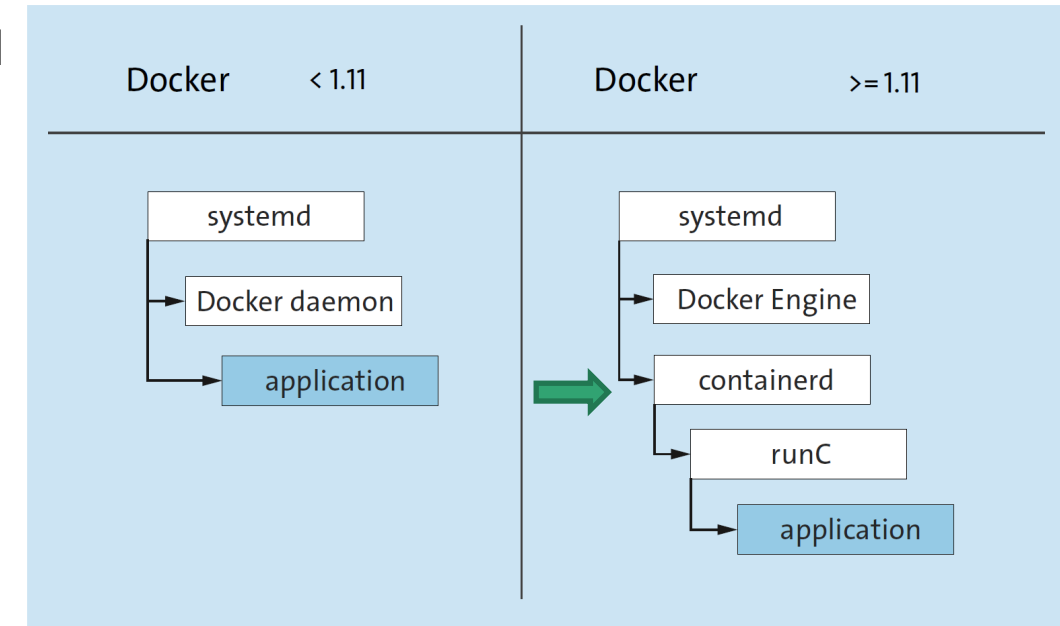
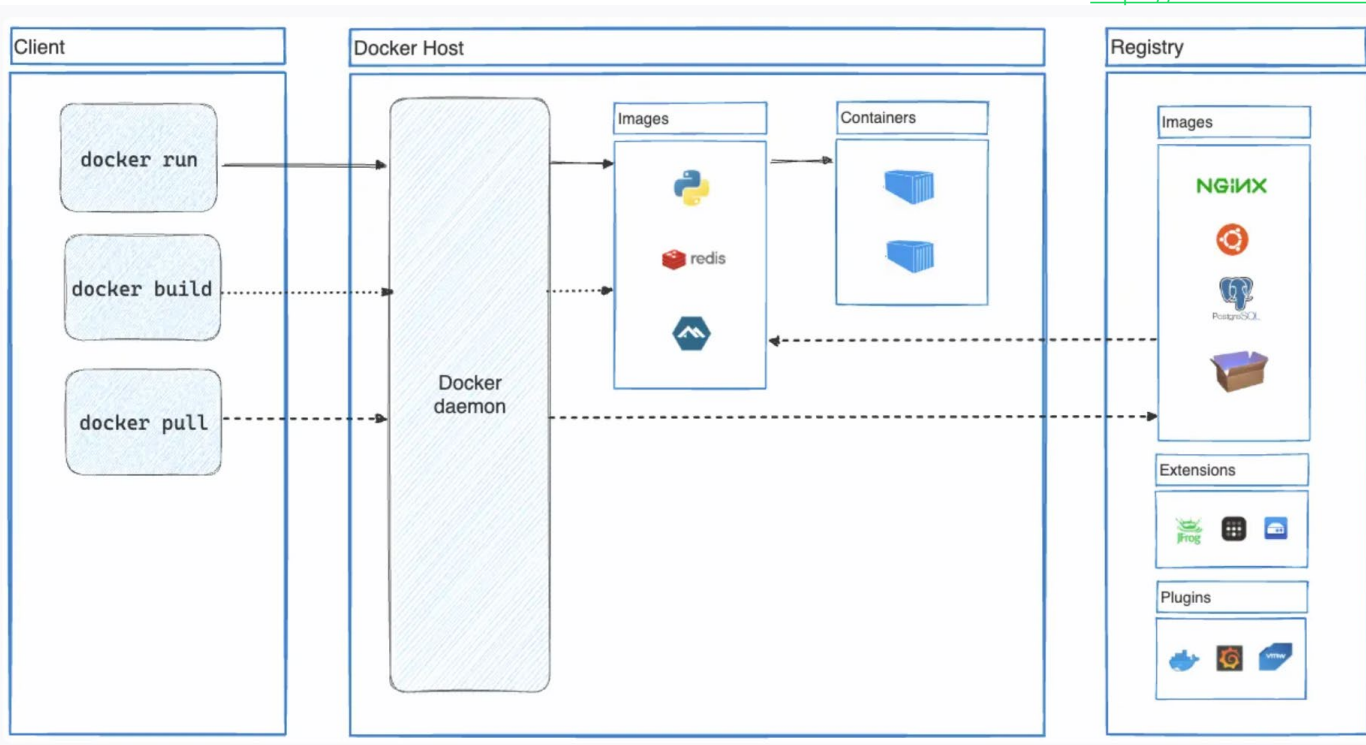


Abbildung 3.4 Funktionaler Docker Stack Pre-/Post 1.11

Container: Funktionaler Überblick

- Konzepte

<https://hub.docker.com>



- ◇ **Registry** – wo die Container Images abgelegt werden.
- ◇ (Docker) **Container Host** – Maschine, i.d.R. eine VM, wo die Container (Prozesse) laufen
- ◇ **Container Image** – paketierte Applikationen / Microservices
- ◇ **Container** – die laufende Applikation / Microservice (Prozess)
- ◇ **Client** – mit docker, kubectl, podman, oc (OpenShift) zum starten der Container

Docker: CLI (1)

- Befehlsaufbau:

◇ `docker <sub-command>`

Container Images bauen, von/nach Registry

- **build:** neues Image erzeugen
- **tag:** Image weitere Bezeichnung zuweisen, z.B. um Registry zu wechseln.
- **pull:** Container Image von Registry holen
- **push:** Container Image in Registry schreiben

Container starten/stoppen etc.

- *run:* startet den Container und erzeugt eine randomisiert oder explizit benannte neue Instanz des Images
- *start:* startet eine vorhandene, gestoppte Container-Instanz
- *stop:* stoppt eine laufende Container-Instanz
- *kill:* Killen eines Containers (gegebenenfalls mit explizitem Signal `[-s]`, Default ist `SIGKILL`)

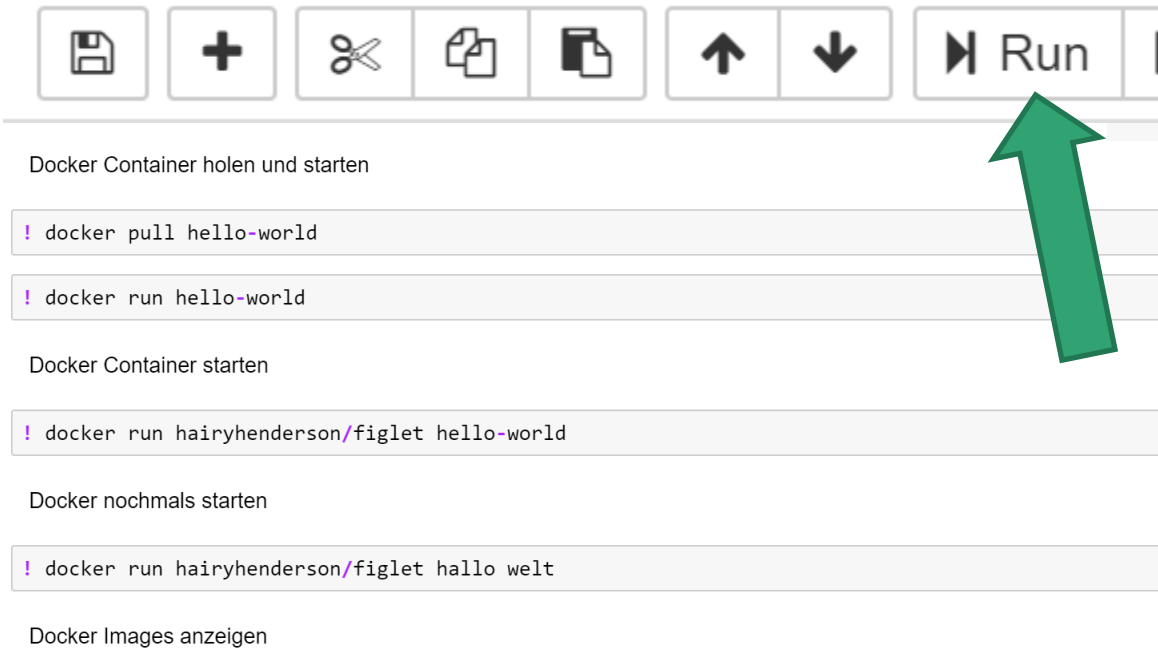
Docker: CLI (2)

Container starten/stoppen etc.

- **rm:** löscht eine gestoppte Container-Instanz und alle damit verbundenen Dateien des Read/Write Layers im unterliegenden Dateisystem
- **ps:** Container-Instanzen auflisten
- **container prune:** (ab 1.13) Container-Instanzen löschen
- **restart:** restartet einen laufenden Container. Per `-t` kann ein maximaler Timeout für Stopp angegeben werden, bevor der Container per `kill` beendet und neu gestartet wird.
- **container:** (ab Version 1.13) neues Subkommando für die Container-spezifischen Verwaltungsbefehle: `[container] attach cp diff export kill ls port rename rm start stop unpause wait commit create exec inspect logs pause prune restart run stats top update`
- **cp:** Dateien zwischen Host und Container kopieren
- **diff:** zeige Änderungen im Filesystem des laufenden Containers
- **exec:** Kommandos innerhalb des Containers ausführen
- Dokumentation:
<https://docs.docker.com/engine/reference/run/>

Übung 03-1: Funktionaler Überblick

- Klickt auf den Link unten und spielt die Übung durch:



The screenshot shows a Jupyter Notebook interface with a toolbar at the top containing icons for saving, adding, deleting, copying, pasting, and running. The 'Run' button is highlighted with a green arrow. Below the toolbar, the notebook contains several cells with Docker commands:

```
Docker Container holen und starten
! docker pull hello-world

Docker Container starten
! docker run hello-world

Docker nochmals starten
! docker run hairyhenderson/figlet hello-world

Docker Images anzeigen
```

- Ergebnis:** Docker besteht aus **Client** und **Docker Host** mit dem zentralen **Docker Daemon**. Der Daemon erstellt und verwaltet **Images** sowie die daraus gestarteten **Container**. Befehle wie `docker run` steuern direkt den Lebenszyklus der Container.
- Erkenntnisse:** Container basieren auf **Images** und sind deren **laufende Instanzen**. Der **Docker Daemon** übernimmt alle Aufgaben rund um Build, Start und Verwaltung. Docker ermöglicht eine **einheitliche, reproduzierbare Laufzeitumgebung** über verschiedene Systeme hinweg.

Wie kommen wir zu Container Images?

- **Holen von Registries:** Öffentliche Container-Registries bieten eine Vielzahl von vorgefertigten Images, die sofort einsatzbereit sind. Diese Images können einfach heruntergeladen und verwendet werden, was den Einstieg in die Containerisierung erheblich erleichtert
- **Selber bauen:** Für spezifische Anforderungen oder massgeschneiderte Anwendungen ist es oft notwendig, eigene Container-Images zu erstellen. Dies geschieht mithilfe eines **Dockerfiles**, das alle Schritte zur Erstellung des Images definiert.
- Beide Ansätze haben ihre Vor- und Nachteile. Das Herunterladen von Images aus Registries spart Zeit und Aufwand, während das Erstellen eigener Images mehr Flexibilität und Kontrolle bietet.
- **ACHTUNG:** Öffentliche Container-Registries können Sicherheitsrisiken bergen, da die Vertrauenswürdigkeit der Quellen oft unklar ist und es weniger Kontrolle über den Inhalt gibt.

Registries



- Default Registry
 - ◇ UI: <https://hub.docker.com>
 - ◇ Registry: index.docker.io
- Cloud Anbieter
 - ◇ [Azure](#), [AWS](#), Google, etc.
 - ◇ [AWS öffentliche Registry](#)
- Weitere
 - ◇ [GitLab](#) (pro Repository!)
 - ◇ [RedHat](#) (Quay.io)
 - ◇ [Nexus3](#) - Maven (Sonatype)
 - ◇ [Harbor](#) - CNCF Projekt
 - ◇ JFrog [Artifactory](#)
- [What is the best alternative](#) to Docker Registry

Registries, Repositories, Tags

- **Container-Image-Name:** `[REGISTRY_HOST[:REGISTRY_PORT]]/REPOSITORY[:TAG]`
- Ein Container-Image-Name folgt einem bestimmten Format, das verschiedene Informationen über das Image enthält:
 - ◇ **REGISTRY_HOST** ist der Hostname oder die IP-Adresse des Registry-Servers. Optional kann nach einem Doppelpunkt : der **REGISTRY_PORT** angegeben werden, der den Port des Servers spezifiziert.
 - ◇ **REPOSITORY:** Dies ist der Name des Repositories innerhalb der Registry, das verschiedene Versionen eines Container-Images gruppiert.
 - ◇ **TAG:** Nach einem optionalen Doppelpunkt : folgt der Tag, der eine spezifische Version oder Variante eines Images innerhalb eines Repositories darstellt. Tags ermöglichen es, verschiedene Iterationen eines Images zu identifizieren, wie z.B. Versionen oder Konfigurationen. Wird kein TAG angegeben wird **latest** verwendet.
- Das Namensschema sorgt für eine standardisierte Art und Weise, Container-Images zu referenzieren und unterstützt die Versionskontrolle von Anwendungen.

Beispiel: Container Image Tags

- 1.0.0 – Release
 - 1.1.0 – Zwischenrelease
 - 1.1.1 – Patch
 - 1.2.0 – Zwischenrelease
 - 1.3.0 – Zwischenrelease
 - 1.3.1 – Patch
 - 1.3.2 – Patch
 - 1.4.0 – Zwischenrelease
 - 2.0.0 – Release
- Tags sind Etiketten, die spezifische Versionen oder Zustände eines Images kennzeichnen.
 - Sie ermöglichen es, verschiedene Versionen einer Anwendung einfach zu identifizieren und zu verwalten.
 - Durch das Verwenden von Tags können Sie leicht zwischen verschiedenen Versionen Ihrer Container-Images wechseln und die Abhängigkeiten Ihrer Anwendungen verwalten und sicherstellen.
 - Beispiel links: MAJOR.MINOR.PATCH
 - Siehe auch: <https://semver.org/>



Übung 03-1: Container Image Tags

- Klickt auf den Link unten und spielt die Übung durch:

Übung: Container Image Tags

Ein Image-Name dient als eindeutiger Bezeichner, der es ermöglicht, spezifische Container-Images in Repositories zu identifizieren und darauf zuzugreifen.

Der Aufbau eines Image-Namens folgt einem klaren Schema, das typischerweise aus mehreren Teilen besteht:

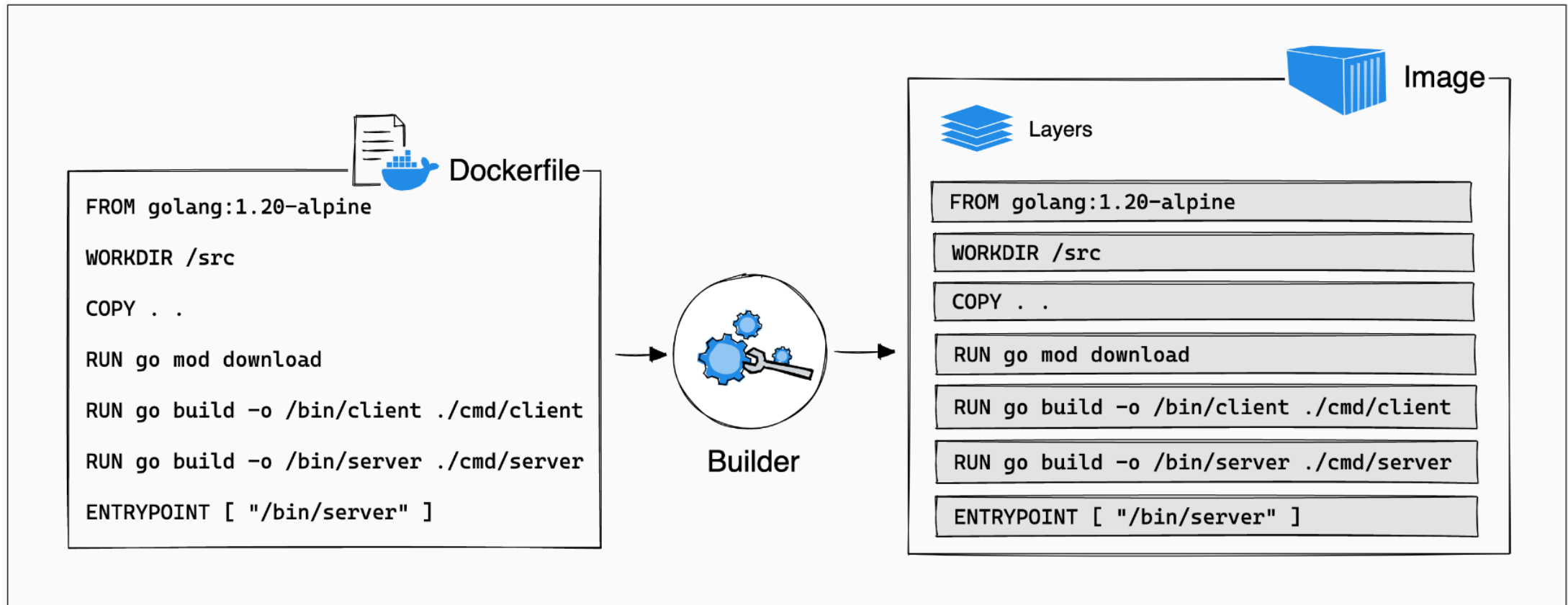
- Registry-Hostnamen
- Repository-Namen
- Tag.

Der **Registry-Hostname** gibt an, wo das Image gespeichert ist, beispielsweise in einer öffentlichen Registry wie Docker Hub oder in einer privaten Registry.

Der **Repository-Name** beschreibt die Anwendung oder den Dienst, den das Image enthält, und organisiert die Images innerhalb der Registry.

Der **Tag** dient der Versionierung und ermöglicht es, verschiedene Versionen eines Images zu unterscheiden und gezielt zu verwenden.

Dockerfile vs. Container Layer



Übung 03-1: Container Image Layers

- Klickt auf den Link unten und spielt die Übung durch:

Übung: Container Layers

Container Layers sind ein zentrales Konzept in der Welt der Containerisierung, das es ermöglicht, Anwendungen effizient und konsistent bereitzustellen.

In der Container-Technologie, insbesondere in Docker, werden Container als eine Abfolge von schreibgeschützten und schreibbaren Layers aufgebaut, die zusammen das vollständige Dateisystem des Containers bilden.

Jeder Layer repräsentiert eine Änderung oder einen Zustand, der auf den vorhergehenden Layers aufbaut, wodurch Änderungen leicht verfolgt und verwaltet werden können.

Dieses Layersystem optimiert nicht nur den Speicherverbrauch, sondern erleichtert auch die Verteilung und Aktualisierung von Anwendungen, indem es redundante Layer minimiert und nur die Unterschiede zwischen den Layers speichert.

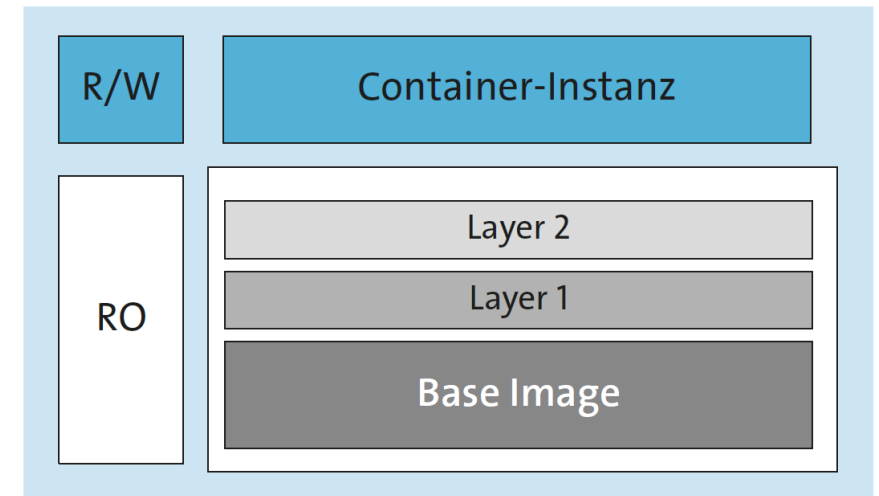


Abbildung 3.9 Schematischer Aufbau eines Images

Container Images bauen und verwalten (1)

- Der effizienteste Weg, um einen bestimmten vordefinierten, funktional validierten und vor allem jederzeit reproduzierbaren (Base-)Image-Stand zu erschaffen, läuft über ein Buildfile, das sogenannte **Dockerfile**.
- Das Dockerfile dient dazu: einen vordefinierten **Build-/Installations-Prozess** zu definieren/starten welcher vollautomatisiert Kommandos durchführt und **an dessen Ende ein fertiges Image** zur Verfügung steht.
- Beim Build-Vorgang selbst wird das per Definition zugrunde liegende Readonly-Basis-Image geklont und in den Speicher geladen, dann wird (pro Build-Aktion) ein neuer Read/Write Layer darauf aufgesetzt. In diesen Read/Write Layern werden die im Dockerfile gelisteten Aktionen abgearbeitet und bei Erfolg (als intermediate Build) committed.
- Schaut man während eines Builds auf die Docker-Prozesse (docker ps), sieht man, dass der Intermediate Build als Container-Prozess auf dem Host aktiv ist.
- **Alternativen:** Google Cloud [buildpacks](#), Getting started with [Buildah](#), [Docker without Docker](#), built-in container support for the [.NET SDK](#), [Dockerfile vs Buildpacks](#).

Dockerfile-Direktiven/-Instruktionen

- **FROM** – von welchem Basisimage abgeleitet
- ~~• **MAINTAINER** – Autor (deprecated).~~
- **RUN** – im Read/Write Layer des Intermediate Container auszuführende Aktionen
- **ENV** – Umgebungsvariablen
- ~~• **EXPOSE** – Portmappings (K8s: Services)~~
- **ENTRYPOINT** – »fixes« Startkommando für primäre Applikation
- **CMD** – überschreibbares Startkommando für primäre Applikation
- ~~• **HEALTHCHECK** – Überwachung des Prozesses im Container auf Funktionstüchtigkeit. (K8s Health Probe Patterns)~~
- ~~• **LABEL** – eindeutige Bezeichnungen für spätere Selektion (K8s Labels).~~
- **ADD** – Files, Directories, remote URLs in Image kopieren
- **COPY** – Files in Image kopieren
- ~~• **VOLUME** – (persistente) Volumes für Image definieren (K8s PersistentVolumesClaim)~~
- **USER** – User für Kommandoausführung festlegen
- **WORKDIR** – Setze das Working Directory für CMD, RUN, COPY, ADD, ENTRYPOINT
- **STOPSIGNAL**: legt das Systemaufrufsignal fest, das zum Beenden an den Container gesendet wird. Z. B. 9, oder ein Signalname im Format SIGNAME, z. B. SIGKILL.

Beispiele (vereinfacht): Dockerfile (1)

- Hello-world Linux

- ◇ FROM scratch # leeres Dateisystem
- ◇ COPY hello /
- ◇ CMD ["/hello"]

- Hello-world Windows

- ◇ FROM
mcr.microsoft.com/**windows**/nano
server
- ◇ COPY hello.txt C:
- ◇ CMD ["cmd", "/C",
"type C:\\hello.txt"]

- Apache Web Server Ubuntu

- ◇ FROM **ubuntu:<version>**
- ◇ RUN apt-get update
- ◇ RUN **apt-get** -q -y install
apache2
- ◇ CMD "/usr/sbin/apache2 -
DFOREGROUND"

- Apache Web Server Fedora

- ◇ FROM **fedora:<version>**
- ◇ RUN **yum** install httpd
- ◇ CMD "/usr/sbin/apache2 -
DFOREGROUND"

Beispiele: Dockerfile (2)

- Alpine Linux – ADD mit gleichzeitigem Entpacken
 - ◇ https://github.com/alpinelinux/docker-alpine/blob/da78fcf5c5da55092aa82a97095274b3df648866/x86_64/Dockerfile
- nginx – Problematik Umleitung stdout, stderr (letzte paar Zeilen im Dockerfile)
 - ◇ <https://github.com/nginxinc/docker-nginx/blob/master/stable/alpine/Dockerfile>
(Docker)
 - ◇ <https://github.com/kubernetes/ingress-nginx/blob/main/rootfs/Dockerfile>
(Kubernetes, [dumb-init](#))
- OpenJDK – mit Anleitung wie das Base Image zu verwenden ist:
 - ◇ https://hub.docker.com/_/openjdk
- .NET Core Docker Samples
 - ◇ <https://github.com/dotnet/docker/blob/master/samples/README.md>

Best Build Practices (teilweise veraltet)

- Container sind **kurzlebig** und jederzeit **reproduzierbar**
- ***Wer** hat es gemacht?* (*MAINTAINER* bzw. *LABEL-Direktive*)
- Wie ist es **bezeichnet**? (*--tag*)
- Verwenden eines eigenen **Build-Ordnerns** pro Template
- Verwendung eines **.dockerignore-Files**
- **Schlanke Images** (Ein Image sollte klein, simpel und so schlank/kompakt wie möglich sein.)
- Nur **ein Prozess** pro Container
- Anzahl der **Layer minimieren**/niedrig halten
- Siehe auch: [Best practices for writing Dockerfiles](#) und [Not every container has an operating system inside](#)

Dockerfile Empfehlungen (des Kursleiters)

- Baut für jede eingesetzte Programmiersprachen ein Base Image: Java, .NET, NodeJS etc.
- Leitet die Microservices von diesem Base Image ab.
- Erstellt Richtlinien für
 - ◇ Container Image Tags!
 - ◇ User und Group-IDs in den Containern
 - ◇ Verwendet keine fixen Verbindungen, z.B. zu Logging Systemen
- Beispiel DOCKERFILE
 - ◇ **FROM <unser Base Image>**
 - ◇ **USER 1001:100**
 - ◇ **COPY <MicroService> .**
 - ◇ **CMD oder ENTRYPOINT**



Übung 03-4: Container Life-cycle (docker)

Applikation erstellen

Dazu erstellen wir zuerst unseren (PHP) Microservice

```
%%bash
cat <<EOF% >index.php
<?php echo '<p>Ich bin ein Microservice</p>'; ?>
EOF%
```

Packetierung

Und für die Paketierung des Microservices, als Container Image, ein Dockerfile.

CMD können wir weglassen, weil im Base Image `php:7-apache` bereit vorhanden.

```
%%bash
cat <<EOF% >Dockerfile
FROM public.ecr.aws/docker/library/php:7.4-apache
COPY index.php /var/www/html/
USER 1000:100
EOF%
```

Beides zusammen ergibt mit `docker build` unser Container Image.

```
%%bash
docker build -t myservice .
```

Das Resultat ist ein neues Image und das, dass Base Image (php) heruntergeladen wurde:

```
! docker image ls
```

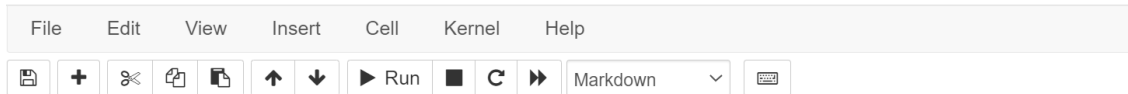
Container starten

- **Alternativ** kann statt Docker Podman verwendet werden.
- Ersetzt den Befehl 'docker' mit 'podman' im Jupyter-Notebook.
- Der Code sieht dann etwa so aus:
 - ◇ `podman build -t myservice .`
 - ◇ `podman image ls`
 - ◇ `podman run --rm -d -p9090:80 \ --name myservice myservice`



Übung 03-5: Container Life-cycle (docker + K8s)

jupyter 03-5-Docker-Life-cycle Last Checkpoint: vor 2 Minuten (autosaved)



Erstellung des Images (Dev)

Dazu erstellen wir zuerst unseren (PHP) Microservice

```
In [ ]: %%bash
cat <<%EOF% >index.php
<?php echo '<p>Hallo Welt</p>'; ?>
%EOF%
```

[.docker.com/](https://www.docker.com/)

Und für die Paketierung des Microservices, als Container Image, ein Dockerfile

```
In [ ]: %%bash
cat <<%EOF% >Dockerfile
FROM php:7.0-apache
COPY index.php /var/www/html/
%EOF%
```

Beides zusammen ergibt mit `docker build` unser Container Image.

```
In [ ]: %%bash
docker build -t index.docker.io/${USER}/myservice .
```

Produktion (Ops)

Das Operation, holt das Container Image von der Registry und startet dieses mittels Kubernetes.

```
In [ ]: ! kubectl run myservice --image=${USER}/myservice --restart=Never
! kubectl expose pod/myservice --type="LoadBalancer" --port 80
! kubectl config view -o=jsonpath='{.clusters[0].cluster.server}' | sed -e 's,
```

Docker Networking



- Wie alles im Container-Universum sind auch die Dinge im Bereich des Networkings für den Container-technisch Unbedarften etwas speziell, zumindest aber gewöhnungs-bedürftig.
- Die DevOps-Teams müssen sich hier etwas von der klassischen Sichtweise »Meine VM und die darin residierenden Services sind immer via IP/Port XYZ zugänglich« verabschieden.
- Das Docker-Netzwerkmodell gilt nur für Docker-Standalone-Nodes und Swarm-Clustern.
- Es ist nicht kompatibel mit Kubernetes.

Docker Networking: bridge, host, none und mehr

- **--net=bridge** – Verwende die Docker Bridge. Der Container kommuniziert mit docker0
- **--net=none** – Kein Netzwerk. Jeder Container ist netzwerktechnisch isoliert.
- **--net=container:<ID>** – Verwende den Netzwerkstack/Netzwerk-Namespace eines anderen Containers. Ähnlich Kubernetes.
- **--net=host** – Verwende das Host-Netzwerk. Der wird Container angewiesen, direkt auf den Netzwerkstack des Hosts unter Umgehung der Bridge zuzugreifen. Sicherheitstechnisch bedenklich!
- **--link** – [Legacy Links](#): Container direkt (beim Start) miteinander verknüpfen (alt)

Docker Compose (K8s: YAML Dateien, HELM etc.)

- **Docker-Compose** ein Tool aus dem Docker-Universum, mit dem die Erzeugung, das Ausrollen und die Verwaltung von multiplen Images/Container-Instanzen, die miteinander interagieren sollen, automatisiert werden kann.
- Über **Compose** und entsprechende, **Yaml-basierte Template-Dateien** können wir unsere containerisierten Applikationen mit einem einzigen Kommando ausrollen.
- Hinweis: [kompose](#) ist ein Werkzeug, um Benutzern, die mit Docker Compose vertraut sind, beim Umzug nach Kubernetes zu helfen. Kompose nimmt eine Docker-Compose-Datei und übersetzt sie in Kubernetes-Ressourcen.



Übung 03-6: kompose

Jupyter 03-6-kompose Last Checkpoint: 4 minutes ago



File Edit View Run Kernel Settings Help

Trusted

Markdown

JupyterLab Python 3 (ipykernel)

Übung: kompose

Docker Compose ist ein leistungsfähiges Werkzeug, das die Orchestrierung von Multi-Container-Docker-Anwendungen vereinfacht.

Anstatt mühsam einzelne Container manuell zu starten und zu konfigurieren, ermöglicht Docker Compose die Definition und Verwaltung von komplexen Anwendungen in einem einzigen YAML-Datei-Format.

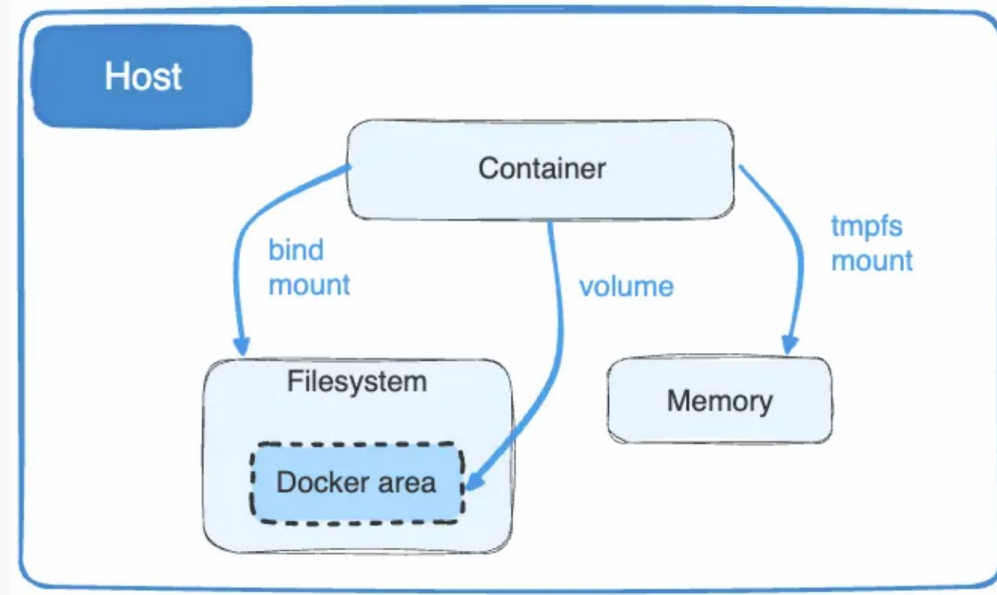
kompose ist ein benutzerfreundliches und leistungsstarkes Werkzeug, das die Konvertierung von Docker Compose-Dateien in Kubernetes-Manifestdateien vereinfacht. Es wurde entwickelt, um Entwicklern den Übergang von Docker-basierten Entwicklungsumgebungen zu Kubernetes-Produktionsumgebungen zu erleichtern.

Zu Übungszwecken wandeln wir eine Docker Compose Datei nach Kubernetes.

Zuerst erstellen wir eine Docker Compose YAML-Datei:

```
[ ]: %%bash
cat <<EOF% >docker-compose.yaml
services:
  customer:
    container_name: customer
    image: registry.gitlab.com/ch-mc-b/autoshop-ms/app/shop/customer:2.0.0
    ports:
      - "8080"
  catalog:
    container_name: catalog
    image: registry.gitlab.com/ch-mc-b/autoshop-ms/app/shop/catalog:2.0.0
    ports:
      - "8080"
```

Docker Persistenz (K8s: PersistentVolume)



- Die Daten bleiben nicht erhalten, wenn ein Container nicht mehr vorhanden ist.
- Es ist schwierig wenn mehrere Container auf die gleichen Daten zugreifen müssen.
- Lösungen
 - ◇ Daten auf den Container Host schreiben
 - ◇ Ein Container Volume einrichten (K8s: Persistentvolume, Storageclass)
- **Perfekt um unterliegende VM zu hacken:**
 - ◇ `docker run -v /:/homeroot -it ubuntu bash`
- Weitere Informationen
 - ◇ <https://docs.docker.com/storage/>



Reflexion

- Container sind **nicht** neu.
- Container ~~ersetzen~~ **ergänzen** Virtualisierung.
- Container, **welcher der «Open Container Initiative Runtime Specification» entsprechen**, sind universell portabel.
- Container sind ~~prinzipiell~~, **bei entsprechendem Handling (keine unnötige SW, nicht privilegiert, ...)**, sicher.
- ~~Container heissen Docker.~~
- **Anmerkung:** Erfahrungen des Kursleiters.

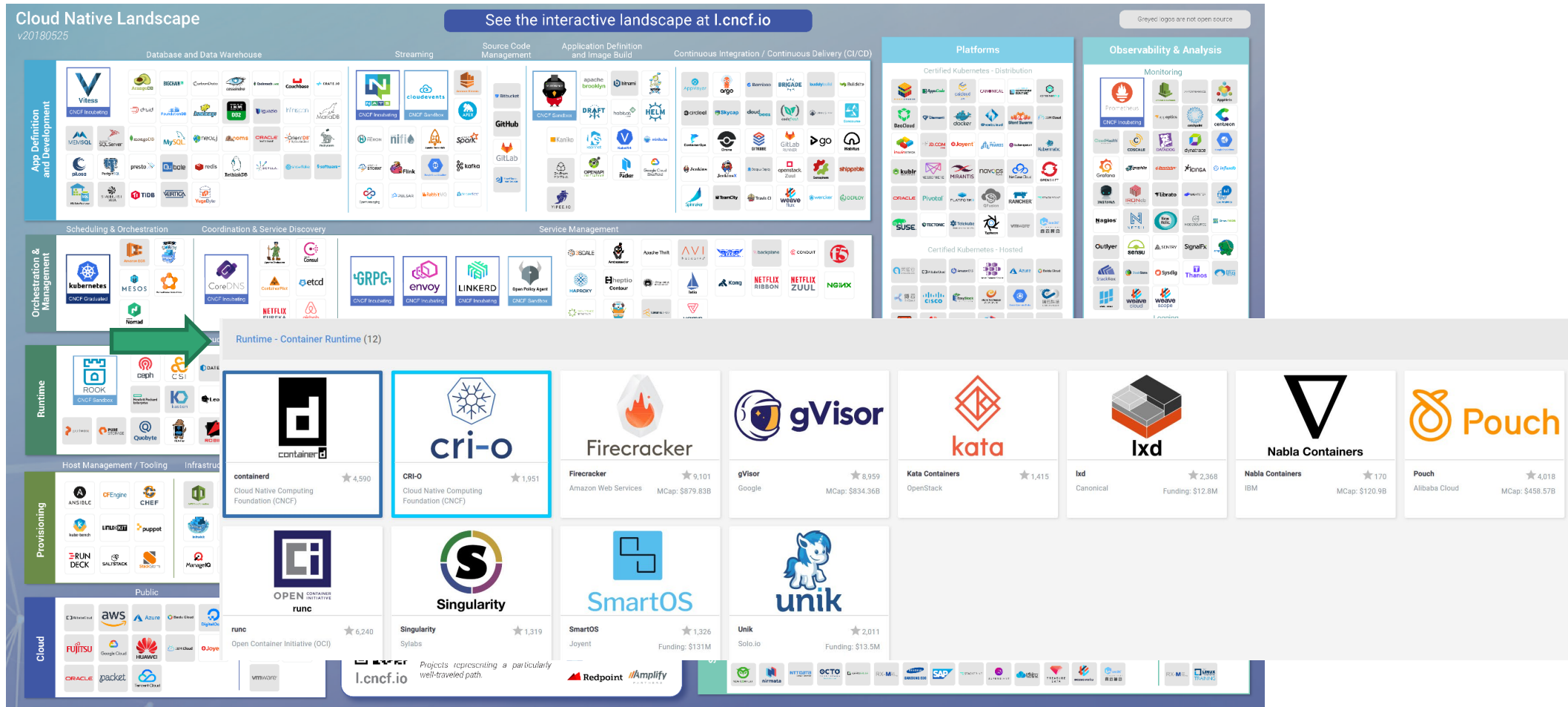


Lernzielkontrolle

- Sie haben einen Überblick über Container Laufzeitumgebungen wie Docker, CRI-O, containerd.

Wo finde ich Container Runtimes?

CNCF Cloud Native Landscape



Quelle: <https://landscape.cncf.io/>

Aufbau eines Images

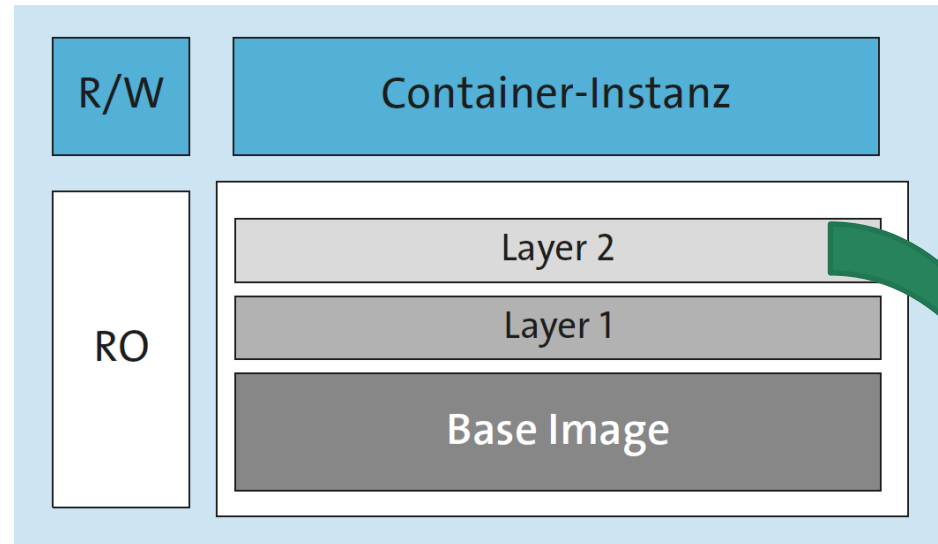
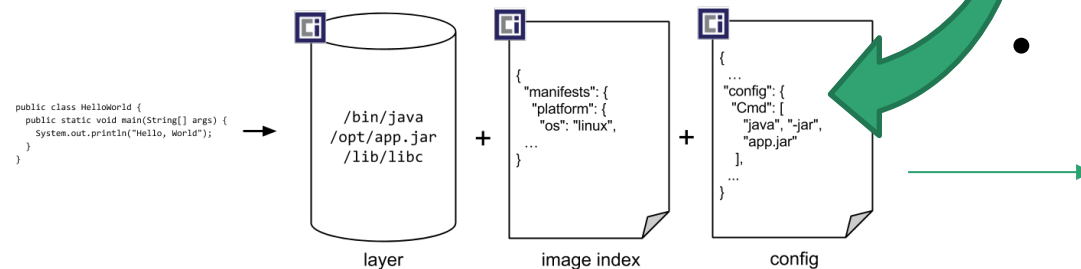


Abbildung 3.9 Schematischer Aufbau eines Images



- Container Instanz: Read / Write
- Container Image: Read Only
 - ◇ Besteht aus Layers (Snapshots), max. 127 (Begrenzung [Overlay2](#))
 - ◇ Jede **Zeile** in Konfigurationsdatei (Dockerfile) erzeugt einen **neuen Layer**.
 - ◇ Als **Base Image** wird der **unterste Layer** bezeichnet. Dieser kann von einem anderen Image abgeleitet oder leer (scratch) sein.
 - ◇ **V2**: Multi-Architektur-Images – Plattformspezifische Versionen
 - ◇ **V2**: Images mit eindeutigem Hash (<https://theupdateframework.io/>)
- Image Standardisierung
 - ◇ <https://www.opencontainers.org/>
 - ◇ <https://github.com/opencontainers>
 - ◇ <https://github.com/opencontainers/image-spec/blob/master/spec.md>

Container-Datei- und Layer-Operationen mit Overlay(FS)

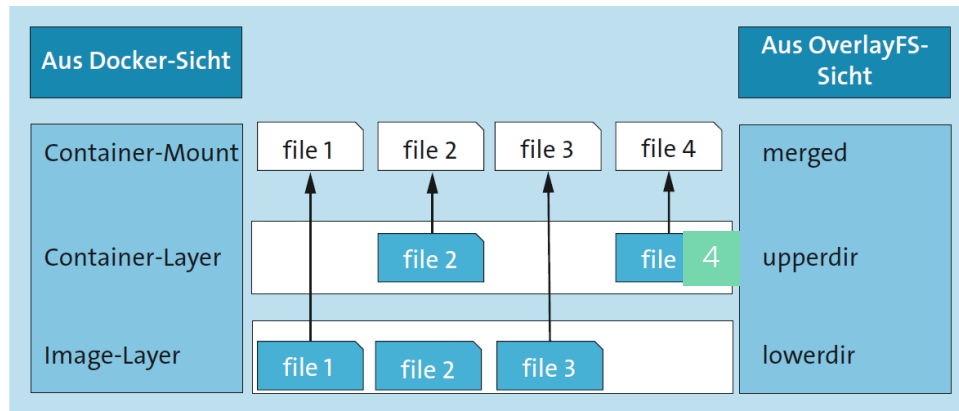


Abbildung 4.11 OverlayFS(2)-Modell

- Die Datei (file1/3) existiert noch nicht im Read/Write-Layer des Containers (upperdir):
 - ◇ Die Datei wird der Image-Ebene (lowerdir) gelesen.
- Die Datei (file4) existiert nur in der Container-(Read/Write-)Ebene (upperdir):
 - ◇ Die Datei wird aus dem Read/Write-Layer des Containers gelesen.
- Die Datei (file2) existiert im Read/Write-Layer des Containers (upperdir) und in der Image-Datei-(Readonly-Layer-)Ebene (lowerdir):
 - ◇ Die Datei wird aus dem Container-Layer gelesen.
- **merged:** Verzeichnis `/var/lib/docker/overlay2`



Übung: Container Life-cycle(mit skaffold - interaktiv)

```

apiVersion: skaffold/v4beta1
kind: Config
metadata:
  name: shop-skaffold
build:
  artifacts:
    - image: localhost:32000/shop/catalog
      context: catalog
      docker:
        dockerfile: Dockerfile
    - image: localhost:32000/shop/customer
      context: customer
      docker:
        dockerfile: Dockerfile
    - image: localhost:32000/shop/order
      context: order
      docker:
        dockerfile: Dockerfile
    - image: localhost:32000/shop/shop
      context: webshop
      docker:
        dockerfile: Dockerfile
  manifests:
    rawYaml:
      - k8s/catalog-deployment.yaml
      - k8s/catalog-service.yaml
      - k8s/customer-deployment.yaml
      - k8s/customer-service.yaml
      - k8s/order-deployment.yaml
      - k8s/order-service.yaml
      - k8s/webshop-deployment.yaml
      - k8s/webshop-ingress.yaml
      - k8s/webshop-service.yaml
  
```

